Name of Your University / College

Department of Computer Science and Engineering

CodeDuel

CodeDuel: A Real-Time 1v1 Programming Battle Platform

A Project Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by: Your Name
Roll No.: Your Roll Number

Under the guidance of: Name of Project Guide

Academic Year 2025–2026

Certificate

This is to certify that the project report entitled **"CodeDuel: A Real-Time 1v1 Programming Battle Platform"** submitted by **Your Name** (Roll No. Your Roll Number) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of Name of Your University / College is a record of bona fide work carried out by the student under my supervision and guidance during the academic year 2025–2026.

The work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Name of Project Guide	**Head of Department**
Project Guide	Department of Computer Science and Engineering

Declaration

I hereby declare that the project work entitled **"CodeDuel: A Real-Time 1v1 Programming Battle Platform"** submitted to Name of Your University / College is a record of original work done by me under the guidance of **Name of Project Guide**. This report has not been submitted, in part or in full, to any other university or institution for the award of any degree, diploma or fellowship. All sources of information used in this report have been duly acknowledged.

Your Name
Roll No. Your Roll Number
Date:

Acknowledgement

I would like to express my sincere gratitude to my project guide, **Name of Project Guide**, for the valuable guidance, constant encouragement and constructive feedback throughout the development of this project.

I am thankful to the Head of the Department and all faculty members of the Department of Computer Science and Engineering for providing the resources and environment necessary to complete this work. I also thank my friends and family for their support, and the open-source communities behind React, Spring Boot, PostgreSQL, Redis, Docker and Judge0, whose tools made this project possible.

Your Name

Abstract

Competitive programming platforms are widely used for learning algorithms, yet most of them are *asynchronous*: a learner solves a problem alone and receives feedback only from an automated judge. This removes the pressure, motivation and social element of real competition. **CodeDuel** is a web-based, real-time platform in which two programmers are matched, receive the same problem, write and execute code simultaneously and compete on correctness and completion time.

The system uses a **React** front end with the Monaco code editor, a **Spring Boot** back end exposing REST APIs and WebSocket/STOMP channels, **PostgreSQL** for persistent data, **Redis** for the matchmaking queue and temporary battle state, and a sandboxed **Judge0** service running in **Docker** containers for safe execution of untrusted code. A rating-aware matchmaking algorithm with a progressively widening rating window pairs players, a finite-state machine controls the life cycle of every battle, an Elo-based rating system updates player skill after each match, and all judging and winner decisions are made server-side to prevent client-side manipulation.

This report presents the requirement analysis, architecture, database design, algorithms, implementation, security measures, testing and evaluation of CodeDuel, together with its limitations and future scope.

Keywords: Real-time systems, WebSocket, Spring Boot, React, Online judge, Matchmaking, Elo rating, Docker sandbox.

Contents

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

1 Introduction 1

1.1 Background . . . 1

1.2 Problem Statement . . . 1

1.3 Objectives . . . 1

1.4 Scope of the Project . . . 2

1.5 Methodology . . . 2

1.6 Organisation of the Report . . . 2

2 Literature Survey and Existing Systems 4

2.1 Overview of Online Judge Systems . . . 4

2.2 Comparison with Existing Platforms . . . 4

2.3 Related Technologies . . . 4

2.4 Research Gap . . . 5

3 System Analysis and Requirements 6

3.1 Functional Requirements . . . 6

3.2 Non-Functional Requirements . . . 7

3.3 Actors and Use Cases . . . 7

3.4 Feasibility Study . . . 8

3.5 Technology Stack . . . 8

4 System Design **9**

4.1 Overall Architecture 9

4.2 Backend Layered Architecture 10

4.3 Battle Life Cycle (State Machine) 10

4.4 Matchmaking Flow 11

4.5 Sequence Diagram: Submission and Judging 12

4.6 Database Design 13

4.6.1 Entity–Relationship Diagram 13

4.6.2 Table Descriptions 13

4.6.3 Use of Redis 14

4.7 API Design 14

4.7.1 REST Endpoints 14

4.7.2 WebSocket Events 15

4.8 User Interface Design 15

5 Implementation **16**

5.1 Project Structure 16

5.2 Authentication with JWT 16

5.3 Matchmaking Algorithm 17

5.4 WebSocket Configuration 18

5.5 Code Execution via Judge0 18

5.6 Winner Determination 19

5.7 Elo Rating System 20

5.8 Frontend Implementation 20

5.9 Deployment 21

6 Security Considerations **22**

7 Testing **23**

7.1 Testing Strategy 23

7.2 Sample Test Cases 23

8 Results and Analysis **24**

8.1 Effort Distribution (Pie Chart) 24

8.2 Language Usage in Battles (Pie Chart) 25

8.3 Judging Time per Language (Bar Chart) 25

8.4 Matchmaking Wait Time (Line Chart) 26

8.5 Real-Time Event Latency (Bar Chart) 26

8.6 Test Results Summary 27

8.7 Discussion 27

8.8 Limitations 27

9 Conclusion and Future Scope **28**

9.1 Conclusion 28

9.2 Future Scope 28

A Project Schedule (Gantt Chart) **29**

B Sample Problem Definition **30**

References **31**

List of Figures

1.1 Incremental development phases 2

3.1 Use case diagram of CodeDuel 7

4.1 High-level system architecture 9
4.2 Layered structure of the Spring Boot backend 10
4.3 Battle state machine 10
4.4 Matchmaking flowchart 11
4.5 Sequence of a code submission 12
4.6 Entity–relationship diagram 13
4.7 Wire-frame of the Battle Room 15

5.1 Winner determination flow 19
5.2 Expected score of player A as a function of rating difference 20
5.3 Deployment view using Docker Compose 21

8.1 Distribution of development effort across project activities 24
8.2 Share of programming languages used in test battles 25
8.3 Average judging time by language and difficulty 25
8.4 Effect of expanding window on matchmaking wait time 26
8.5 Average WebSocket event delivery latency 26

A.1 Project timeline (16 weeks) 29

List of Tables

2.1 Feature comparison with existing platforms 4

3.1 Functional requirements 6

3.2 Non-functional requirements 7

3.3 Technology stack 8

4.1 Battle states 11

4.2 Database tables 13

4.3 Redis data usage 14

4.4 Main REST API endpoints 14

4.5 WebSocket events on channel `/topic/battle/{battleId}` 15

5.1 Expanding rating window 17

6.1 Threats and mitigations 22

7.1 Sample test cases 23

8.1 Summary of testing results (sample data) 27

Chapter 1

Introduction

1.1 Background

Programming skill is one of the most sought-after competencies in the software industry. Learners typically practise on online judges such as LeetCode, Codeforces and HackerRank. While these platforms provide huge problem libraries, practice is largely solitary. Live contests exist but are scheduled events with hundreds or thousands of participants, and a beginner may feel lost in such a crowd.

A *one-versus-one* format offers a middle path: short, personal, high-engagement competition available on demand. Two players enter the same virtual room, see the same problem and race to produce a correct solution. The opponent's progress is visible, but their source code is not, creating tension without enabling copying.

1.2 Problem Statement

Existing practice platforms lack an on-demand, real-time, head-to-head competitive experience with fair, secure server-side evaluation. The problem addressed by this project is:

> *To design and implement a scalable web platform that automatically matches two programmers, gives them the same problem, executes their submissions securely against hidden test cases, and determines the winner in real time while maintaining player ratings.*

1.3 Objectives

- Provide secure registration, login and profile management using JWT authentication.
- Implement an automatic, rating-aware matchmaking system.

- Build a real-time battle room with a shared timer, live opponent status and instant result delivery.
- Execute untrusted user code safely in an isolated, resource-limited sandbox.
- Evaluate submissions against visible and hidden test cases and determine the winner objectively.
- Maintain player ratings with an Elo-style algorithm and display leaderboards and battle history.
- Offer Practice Mode and Private Battles for learning and easy demonstration.

1.4 Scope of the Project

In scope: 1v1 battles, matchmaking, private rooms, practice mode, multi-language execution (Java, Python, C++), ratings, leaderboards, battle history and server-side judging.

Out of scope (future work): team battles, tournaments, video/voice chat, plagiarism detection using machine learning, and mobile native applications.

1.5 Methodology

An *incremental* development model was followed. The project was divided into four phases (Figure 1.1), where each phase yields a working, testable product. This avoids the risk of building an over-large system that is never completed.

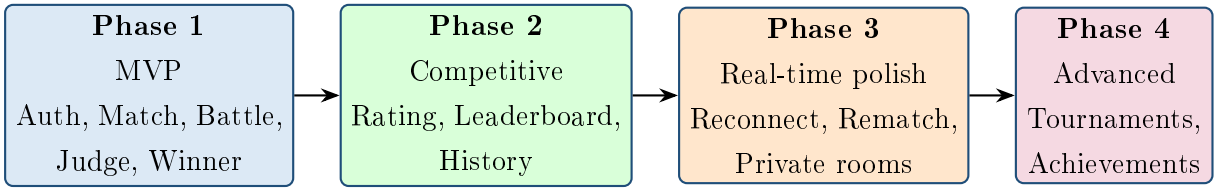


Figure 1.1: Incremental development phases

1.6 Organisation of the Report

Chapter 2 surveys related work. Chapter 3 presents requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation and core algorithms.

Chapter 6 discusses security. Chapter 7 covers testing, and Chapter 8 evaluates results. Chapter 9 concludes with future scope.

Chapter 2

Literature Survey and Existing Systems

2.1 Overview of Online Judge Systems

An online judge compiles and runs submitted code against test cases and returns a verdict such as *Accepted*, *Wrong Answer* or *Time Limit Exceeded*. Popular systems include LeetCode, Codeforces, HackerRank and CodeChef. Open-source judge engines such as Judge0 expose the same capability through an API, enabling developers to embed code execution in their own products.

2.2 Comparison with Existing Platforms

Feature	**LeetCode**	**Codeforces**	**HackerRank**	**CodeDuel**
On-demand 1v1 battles	No	No	No	**Yes**
Live opponent status	No	Scoreboard only	Scoreboard only	**Yes**
Rating-based matchmaking	No	Contest rating	No	**Yes**
Private battle rooms	No	Limited	Limited	**Yes**
Practice mode	Yes	Yes	Yes	**Yes**
Server-side hidden tests	Yes	Yes	Yes	**Yes**
Open-source / self-hostable	No	No	No	**Yes**

Table 2.1: Feature comparison with existing platforms

2.3 Related Technologies

- **WebSocket / STOMP:** Full-duplex, low-latency communication over a single TCP connection, avoiding the overhead of repeated HTTP polling.
- **Elo rating system:** Statistical model, originally for chess, that estimates relative skill from match outcomes.

- **Container sandboxing:** Linux namespaces and cgroups isolate processes and limit CPU, memory and network access.
- **Redis:** In-memory data-structure store suited to queues, presence data and Pub/Sub.

2.4 Research Gap

Existing systems either support only asynchronous practice or large scheduled contests. There is no lightweight, open, on-demand 1v1 platform combining matchmaking, real-time synchronisation and secure judging, which CodeDuel aims to provide.

Chapter 3

System Analysis and Requirements

3.1 Functional Requirements

ID	Requirement
FR1	Users shall register, log in, log out and view/edit their profile.
FR2	The system shall authenticate every request using JWT tokens.
FR3	A user shall be able to join the matchmaking queue and cancel the search.
FR4	The system shall match two waiting players, preferring similar ratings.
FR5	Both players shall receive the same problem and a synchronised countdown/timer.
FR6	Players shall write code in a browser editor and choose Java, Python or C++.
FR7	Players shall be able to *Run* code on sample tests and *Submit* for full judging.
FR8	The system shall display opponent status and test-pass count, but never opponent code.
FR9	The server shall determine the winner and update ratings automatically.
FR10	Users shall be able to create/join private battles using a shareable code.
FR11	The system shall provide practice mode, battle history and a leaderboard.

Table 3.1: Functional requirements

3.2 Non-Functional Requirements

Attribute	Requirement
Performance	Real-time events delivered in under 200 ms; submission verdict within a few seconds.
Security	Passwords hashed (BCrypt); untrusted code sandboxed; hidden tests never exposed.
Scalability	Stateless back end; matchmaking queue and presence stored in Redis.
Reliability	Graceful handling of disconnects; battle state recoverable.
Usability	Clean responsive interface usable on modern browsers.
Maintainability	Layered architecture (controller, service, repository) with clear separation.
Portability	Entire stack deployable with Docker Compose.

Table 3.2: Non-functional requirements

3.3 Actors and Use Cases

Figure 3.1 shows the primary actors and their interactions with the system.

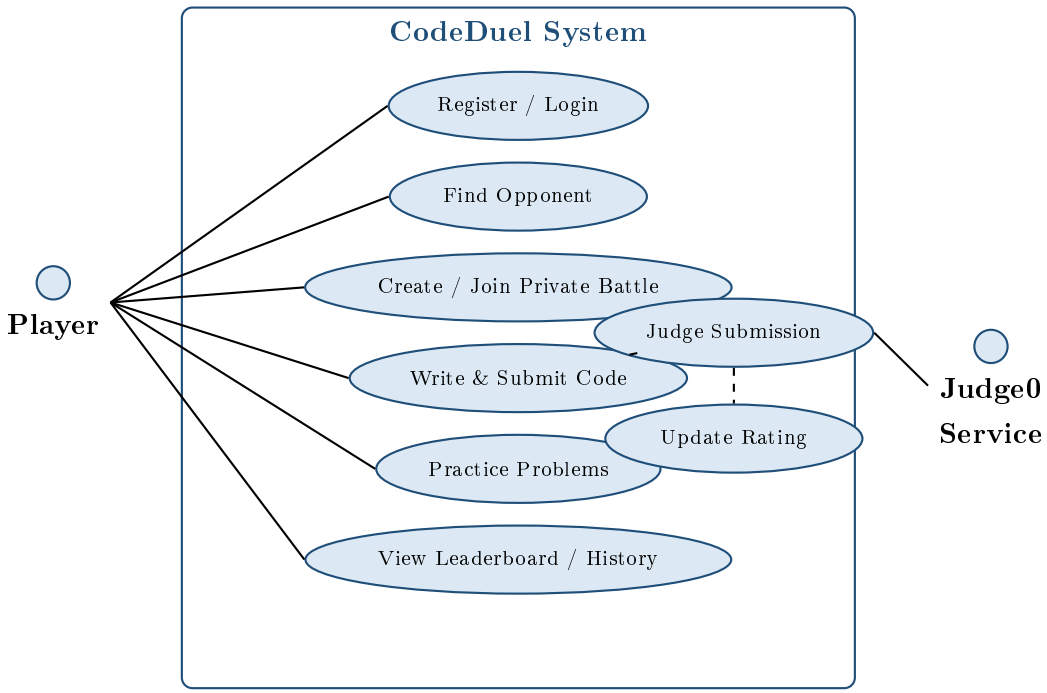


Figure 3.1: Use case diagram of CodeDuel

3.4 Feasibility Study

Technical feasibility: All technologies are mature, open source and well documented. **Economic feasibility:** Free tiers of hosting, Docker and open-source software keep cost minimal. **Operational feasibility:** The interface follows familiar coding-platform conventions, requiring no special training.

3.5 Technology Stack

Layer	Technology	Purpose
Front end	React, Tailwind CSS	Component-based responsive UI
Code editor	Monaco Editor	VS Code editing experience in browser
Back end	Spring Boot (Java)	REST APIs, business logic
Real-time	WebSocket + STOMP	Bidirectional battle events
Security	Spring Security, JWT	Authentication and authorisation
Database	PostgreSQL, JPA	Persistent relational data
Cache / queue	Redis	Matchmaking queue, presence, battle state
Code execution	Judge0	Sandboxed compilation and execution
Container	Docker, Compose	Isolation and deployment
Tools	Maven, Git, Postman, Swagger	Build, VCS, API testing, documentation

Table 3.3: Technology stack

Chapter 4

System Design

4.1 Overall Architecture

CodeDuel follows a client–server architecture with a layered Spring Boot back end. The React client uses REST for request/response operations and WebSocket for real-time events. User code is never executed in the application server; it is forwarded to the isolated Judge0 service.

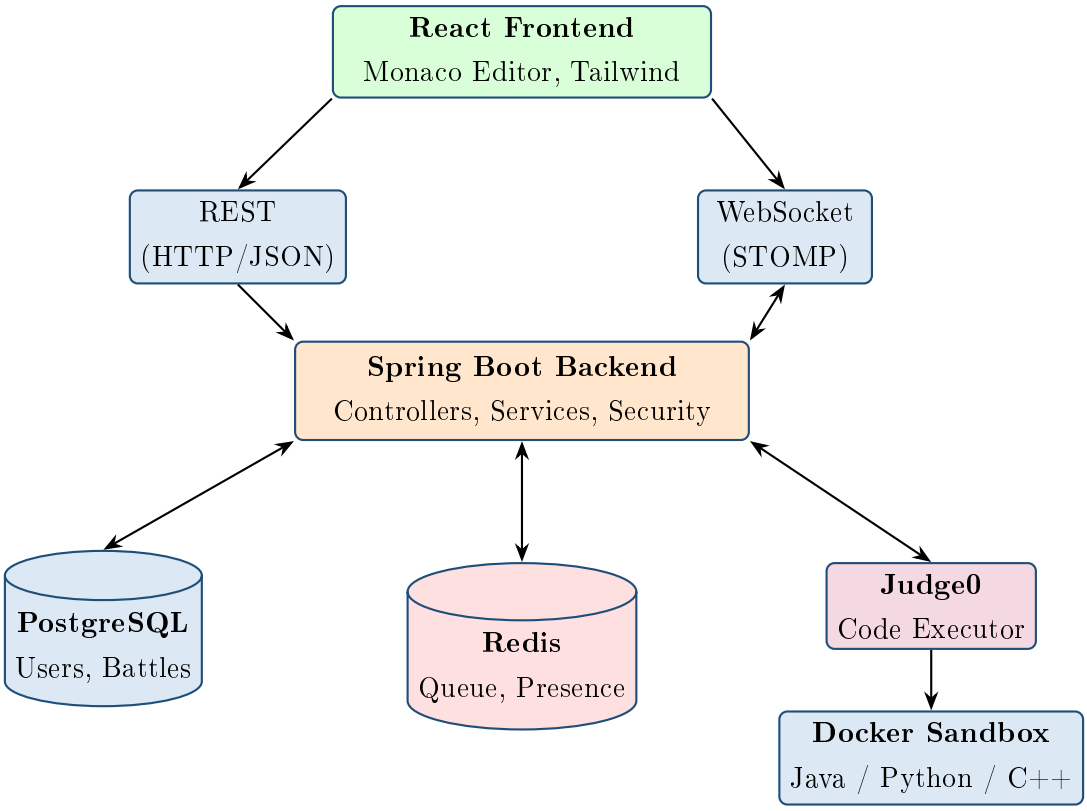


Figure 4.1: High-level system architecture

4.2 Backend Layered Architecture

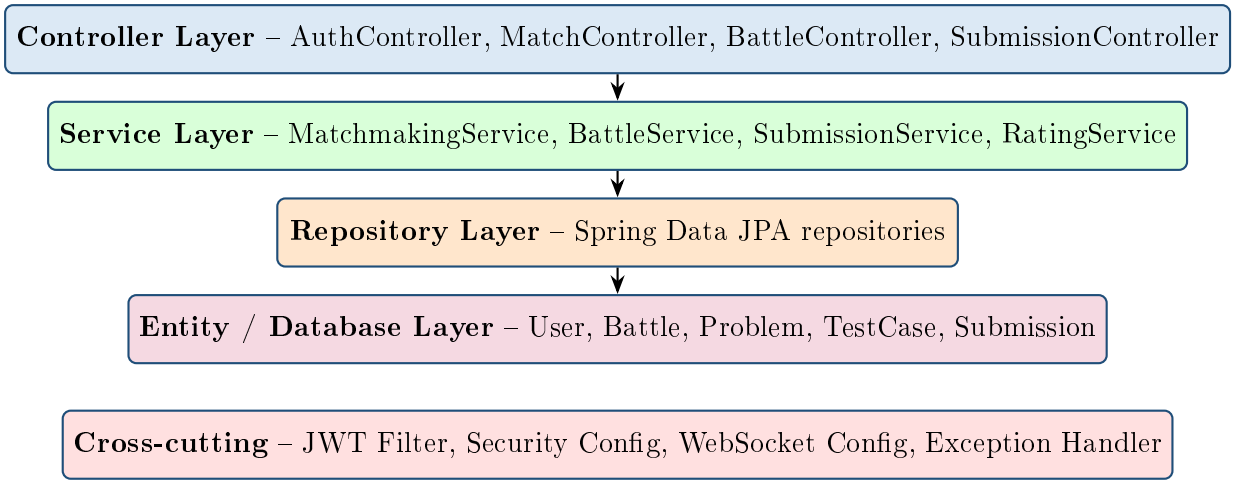


Figure 4.2: Layered structure of the Spring Boot backend

4.3 Battle Life Cycle (State Machine)

Each battle is modelled as a finite-state machine. State transitions occur only on the server, guaranteeing consistency between both players.

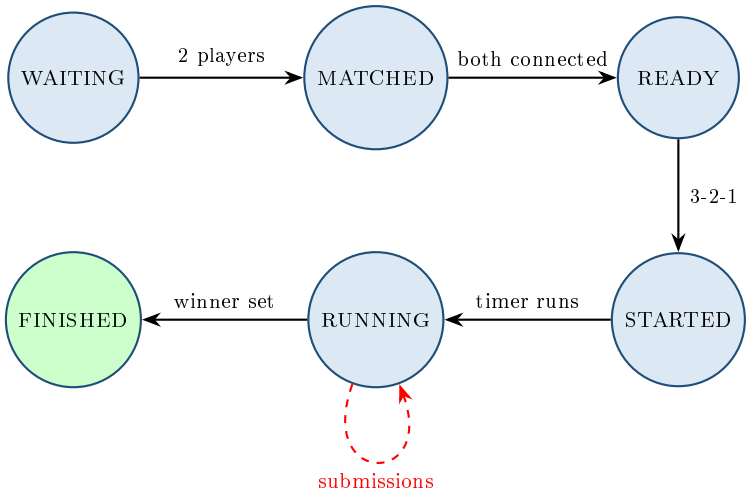


Figure 4.3: Battle state machine

State	Meaning	Trigger
WAITING	Player is in queue or private room is open	Find Opponent / Create room
MATCHED	Two players paired, battle record created	Matchmaker
READY	Both WebSocket connections established	Client join events
STARTED	Countdown finished; problem revealed	Server timer
RUNNING	Players coding and submitting	Submissions
FINISHED	Winner decided, ratings updated	All accepted / time up

Table 4.1: Battle states

4.4 Matchmaking Flow

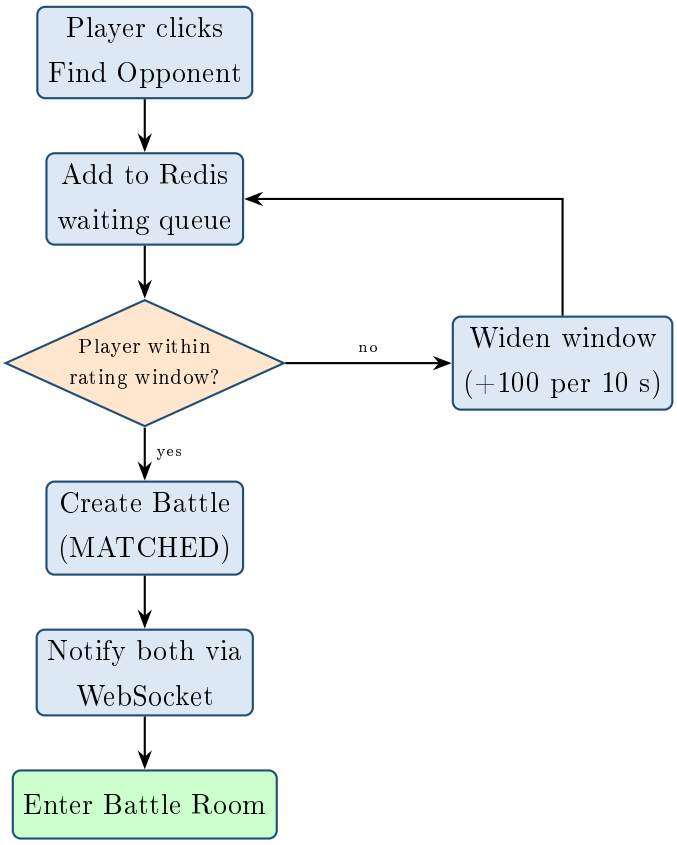


Figure 4.4: Matchmaking flowchart

4.5 Sequence Diagram: Submission and Judging

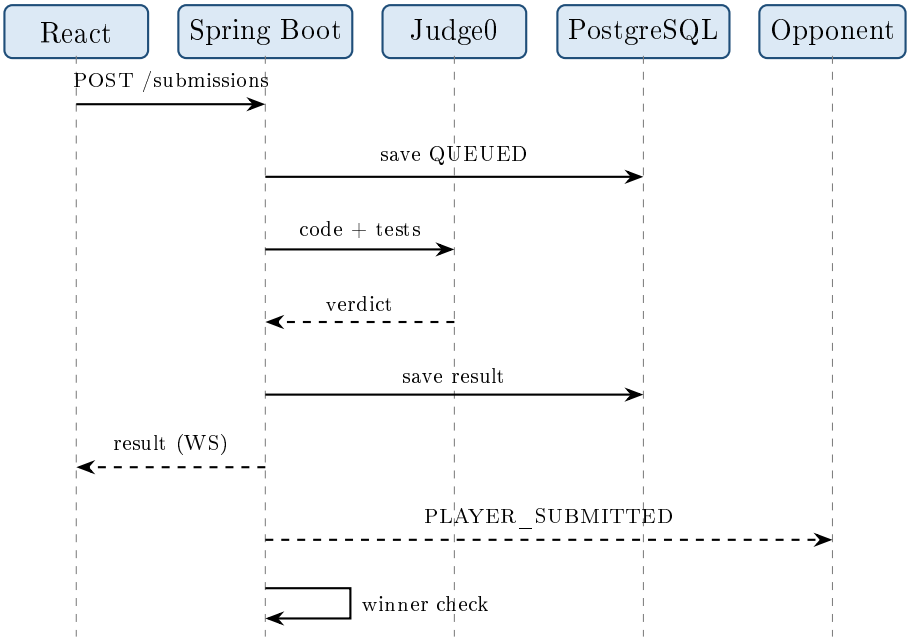


Figure 4.5: Sequence of a code submission

4.6 Database Design

4.6.1 Entity–Relationship Diagram

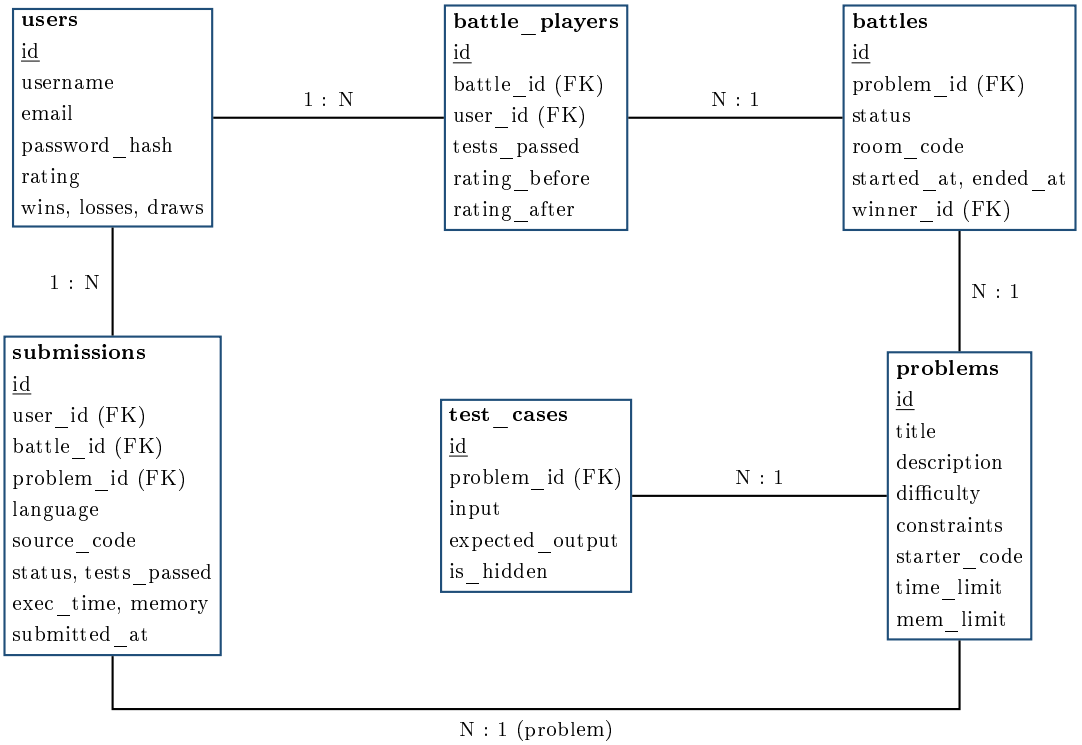


Figure 4.6: Entity–relationship diagram

4.6.2 Table Descriptions

Table	Description
users	Account credentials, current rating and win/loss/draw statistics.
problems	Problem statements, difficulty, constraints, starter code and limits.
test_cases	Visible and hidden input/expected-output pairs for each problem.
battles	One row per match: status, problem, timing and winner.
battle_players	Join table linking two users to a battle with per-player results and rating change.
submissions	Every run/submit with language, verdict, tests passed, time and memory.

Table 4.2: Database tables

4.6.3 Use of Redis

Key pattern	Type	Purpose
`waiting_players`	Sorted set (score = rating)	Matchmaking queue
`battle:{id}`	Hash	Temporary live battle state
`player:{id}`	String (TTL)	Online / presence status
`battle:events`	Pub/Sub channel	Cross-instance event broadcast

Table 4.3: Redis data usage

4.7 API Design

4.7.1 REST Endpoints

Method	Endpoint	Description
POST	`/api/auth/register`	Create a new user account
POST	`/api/auth/login`	Authenticate and return JWT
GET	`/api/profile`	Current user's profile and statistics
GET	`/api/problems/{id}`	Fetch problem (visible tests only)
POST	`/api/matchmaking/join`	Join matchmaking queue
POST	`/api/battles/private`	Create private battle, returns code
POST	`/api/submissions`	Submit code for judging
GET	`/api/leaderboard`	Top players by rating
GET	`/api/battles/history`	Past battles of the user

Table 4.4: Main REST API endpoints

4.7.2 WebSocket Events

Event	Direction	Description
PLAYER_JOINED	Server → Clients	Opponent entered the room
BATTLE_STARTED	Server → Clients	Countdown ended, problem shown
PLAYER_SUBMITTED	Server → Opponent	Opponent submitted; tests passed count
PLAYER_DISCONNECTED	Server → Opponent	Opponent lost connection
TIMER_SYNC	Server → Clients	Authoritative remaining time
BATTLE_FINISHED	Server → Clients	Winner, scores, rating changes

Table 4.5: WebSocket events on channel `/topic/battle/{battleId}`

4.8 User Interface Design

Figure 4.7 shows the wire-frame of the battle room, the core screen of the application.

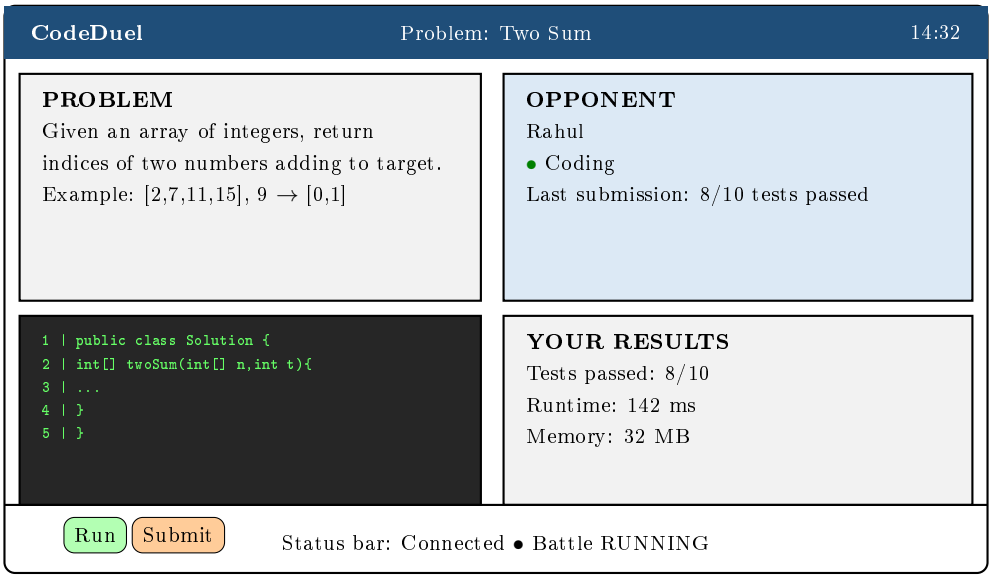


Figure 4.7: Wire-frame of the Battle Room

Chapter 5

Implementation

5.1 Project Structure

```
codeduel/
|-- backend/ (Spring Boot)
|   |-- controller/  AuthController, MatchController,
   BattleController ...
|   |-- service/     MatchmakingService, BattleService, RatingService
    ...
|   |-- repository/  UserRepository, BattleRepository, ...
|   |-- entity/      User, Battle, Problem, TestCase, Submission
|   |-- websocket/   WebSocketConfig, BattleWebSocketController
|   `-- security/    JwtAuthenticationFilter, SecurityConfig
`-- frontend/ (React)
    |-- components/  CodeEditor, ProblemPanel, Timer, PlayerCard ...
    |-- pages/       Login, Home, Matchmaking, Battle, Results,
       Profile
    |-- services/    api.js, auth.js, websocket.js
    `-- context/     AuthContext
```

Listing 5.1: Backend and frontend structure

5.2 Authentication with JWT

Passwords are hashed using BCrypt. On login the server issues a signed JSON Web Token, which the client sends in the `Authorization: Bearer` header. A filter validates the token on every request and also secures the WebSocket handshake.

```
1 public class JwtAuthenticationFilter extends OncePerRequestFilter {
2   @Override
3   protected void doFilterInternal(HttpServletRequest req,
4         HttpServletResponse res, FilterChain chain)
5         throws ServletException, IOException {
6     String header = req.getHeader("Authorization");
7     if (header != null && header.startsWith("Bearer ")) {
8       String token = header.substring(7);
```

```
      if (jwtService.isValid(token)) {
        String username = jwtService.extractUsername(token);
        UserDetails user = userDetailsService.loadUserByUsername(
            username);
        var auth = new UsernamePasswordAuthenticationToken(
                user, null, user.getAuthorities());
        SecurityContextHolder.getContext().setAuthentication(auth);
      }
    }
    chain.doFilter(req, res);
  }
}
```

Listing 5.2: Simplified JWT authentication filter

5.3 Matchmaking Algorithm

Players wait in a Redis sorted set keyed by rating. A scheduled task runs every second. For each waiting player the acceptable rating difference grows with waiting time:

$$\Delta_{\max}(t) = 100 \times \left(\left\lfloor \frac{t}{10} \right\rfloor + 1 \right),$$

where t is the waiting time in seconds. A pair is created when $|R_A - R_B| \leq \min(\Delta_{\max}(t_A), \Delta_{\max}(t_B))$ or, to reduce waiting, the maximum of both.

Waiting time (s)	Allowed rating difference
0–10	±100
10–20	±200
20–30	±300
> 30	±400 or any available player

Table 5.1: Expanding rating window

```
@Scheduled(fixedDelay = 1000)
public void matchPlayers() {
  Set<ZSetOperations.TypedTuple<String>> players =
      redis.opsForZSet().rangeWithScores("waiting_players", 0, -1);
  List<Candidate> list = toCandidates(players);   // id, rating,
      joinedAt
  for (Candidate a : list) {
    int window = 100 * ((int)(a.waitSeconds() / 10) + 1);
    for (Candidate b : list) {
```

```
      if (a.id().equals(b.id())) continue;
      if (Math.abs(a.rating() - b.rating()) <= window) {
        redis.opsForZSet().remove("waiting_players", a.id(), b.id());
        battleService.createBattle(a.id(), b.id());
        return;
      }
    }
  }
}
```

Listing 5.3: Matchmaking scheduler (simplified)

5.4 WebSocket Configuration

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements
    WebSocketMessageBrokerConfigurer {
  @Override
  public void registerStompEndpoints(StompEndpointRegistry registry)
      {
    registry.addEndpoint("/ws")
            .setAllowedOriginPatterns("*").withSockJS();
  }
  @Override
  public void configureMessageBroker(MessageBrokerRegistry registry)
      {
    registry.enableSimpleBroker("/topic");
    registry.setApplicationDestinationPrefixes("/app");
  }
}
```

Listing 5.4: STOMP over WebSocket configuration

5.5 Code Execution via Judge0

When a player submits, the server creates one Judge0 request per test case (or a batch) containing the source code, language id, stdin and expected output, along with CPU-time and memory limits. Results are collected, counted and stored. Untrusted code never runs on the application server.

```
public SubmissionResult judge(Submission s, List<TestCase> tests) {
  int passed = 0;
```

```
    for (TestCase t : tests) {
      JudgeResponse r = judge0Client.execute(
          s.getSourceCode(), s.getLanguageId(),
          t.getInput(), t.getExpectedOutput(),
          problem.getTimeLimit(), problem.getMemoryLimit());
      if (r.getStatus() == Status.ACCEPTED) passed++;
      else if (r.isCompileError()) return SubmissionResult.compileError
          (r);
    }
    s.setTestsPassed(passed);
    s.setStatus(passed == tests.size() ? ACCEPTED : WRONG_ANSWER);
    return repository.save(s);
}
```

Listing 5.5: Submission judging (simplified)

5.6 Winner Determination

The winner is not simply the first to submit. The decision uses a lexicographic comparison:

1. Higher number of test cases passed.
2. If equal, earlier time of the best submission.
3. (Optional extension) If still equal, lower execution time.
4. If all are equal, the result is a draw.

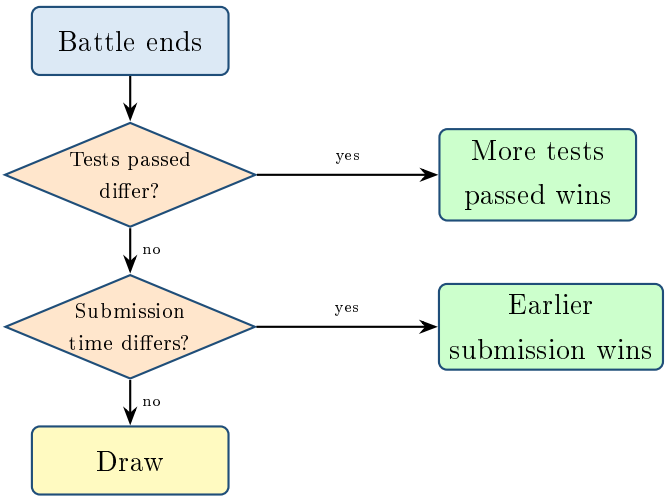


Figure 5.1: Winner determination flow

5.7 Elo Rating System

Let R_A and R_B be the ratings of players A and B. The expected score of A is

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}},$$

and the updated rating is

$$R'_A = R_A + K\,(S_A - E_A),$$

where $S_A \in \{1, 0.5, 0\}$ for a win, draw or loss and K is the development constant (here $K = 32$).

Worked example. Aryan (1250) defeats Rahul (1280): $E_A = 1/(1 + 10^{30/400}) \approx 0.457$. Then $R'_A = 1250 + 32(1 - 0.457) \approx 1267$ and $R'_B = 1280 + 32(0 - 0.543) \approx 1263$.

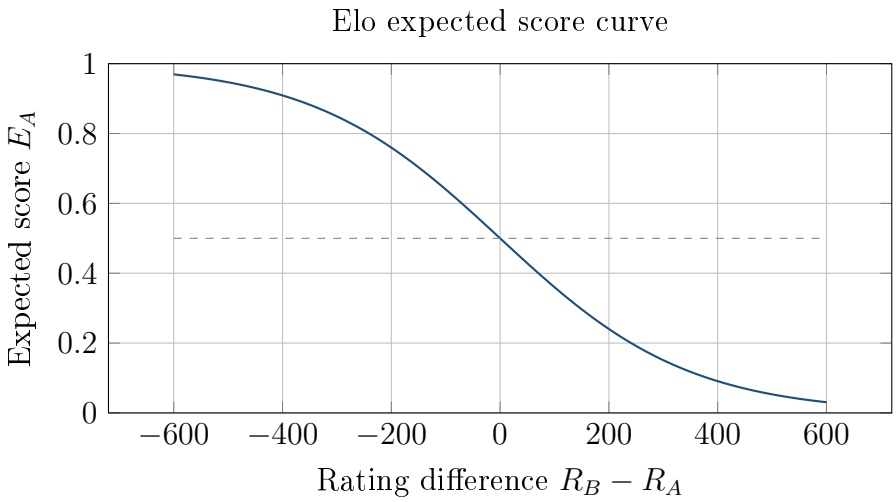


Figure 5.2: Expected score of player A as a function of rating difference

5.8 Frontend Implementation

The React application is divided into pages (Login, Home, Matchmaking, Battle, Results, Profile, Leaderboard) and reusable components (CodeEditor, ProblemPanel, Timer, PlayerCard, SubmissionResult). The Monaco editor is embedded through a wrapper component; a `websocket.js` service manages the STOMP connection, subscribes to `/topic/battle/{id}` and dispatches events to the UI state. An `AuthContext` stores the JWT and the current user.

```
// services/websocket.js
const client = new Client({ brokerURL: WS_URL,
  connectHeaders: { Authorization: `Bearer ${token}` } });
```

```
client.onConnect = () => {
  client.subscribe(`/topic/battle/${battleId}`, msg => {
    const event = JSON.parse(msg.body);
    dispatch(event);   // PLAYER_JOINED, BATTLE_STARTED, ...
  });
};
client.activate();
```

Listing 5.6: Client-side STOMP subscription (JavaScript)

5.9 Deployment

The entire system is containerised and started with Docker Compose.

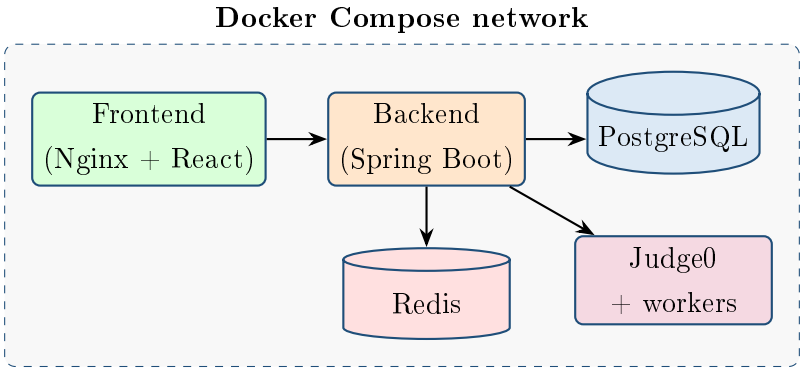


Figure 5.3: Deployment view using Docker Compose

Chapter 6

Security Considerations

Threat	Countermeasure
Malicious code (fork bombs, file access)	Executed only inside Judge0/Docker with CPU, memory, process and time limits; network disabled.
Stolen credentials	BCrypt password hashing; short-lived JWT; HTTPS in production.
Client tampering with result/timer	Client never decides winner; server owns timer and verdict.
Hidden-test leakage	Hidden test cases never sent to client; only pass counts returned.
Opponent code exposure	Only status and pass count broadcast over WebSocket.
Submission spam / DoS	Rate limiting per user; repeated identical submissions detected.
Unauthorised battle access	WebSocket handshake authenticated; user must be a participant of the battle.
SQL injection	Parameterised queries via Spring Data JPA.

Table 6.1: Threats and mitigations

Principle: *The client is untrusted. All decisions about correctness, time and winner are made on the server.*

Chapter 7

Testing

7.1 Testing Strategy

- **Unit testing:** JUnit/Mockito for services (rating, winner logic, matchmaking).
- **Integration testing:** Spring Boot Test with a test PostgreSQL/Redis container.
- **API testing:** Postman collections for all REST endpoints.
- **WebSocket testing:** Two simultaneous browser sessions and scripted STOMP clients.
- **Security testing:** Invalid tokens, tampered payloads, infinite-loop and memory-bomb submissions.

7.2 Sample Test Cases

ID	Module	Test description / expected result	Status
TC01	Auth	Register with valid data creates a user	Pass
TC02	Auth	Login with wrong password returns 401	Pass
TC03	Auth	Request without JWT is rejected	Pass
TC04	Matchmaking	Two players within range are matched	Pass
TC05	Matchmaking	Window widens after 10 s and matches distant players	Pass
TC06	Battle	Both players receive BATTLE_STARTED together	Pass
TC07	Judge	Correct solution returns ACCEPTED with all tests passed	Pass
TC08	Judge	Infinite loop terminates with TIME_LIMIT	Pass
TC09	Judge	Syntax error returns COMPILATION_ERROR	Pass
TC10	Winner	More tests passed beats earlier submission	Pass
TC11	Rating	Winner's rating increases, loser's decreases	Pass
TC12	Security	Opponent source code is not present in any WS message	Pass

Table 7.1: Sample test cases

Chapter 8

Results and Analysis

Note: The numerical values in the charts of this chapter are *illustrative sample data*. Replace them with the measurements you actually obtain from your own testing before submission.

8.1 Effort Distribution (Pie Chart)

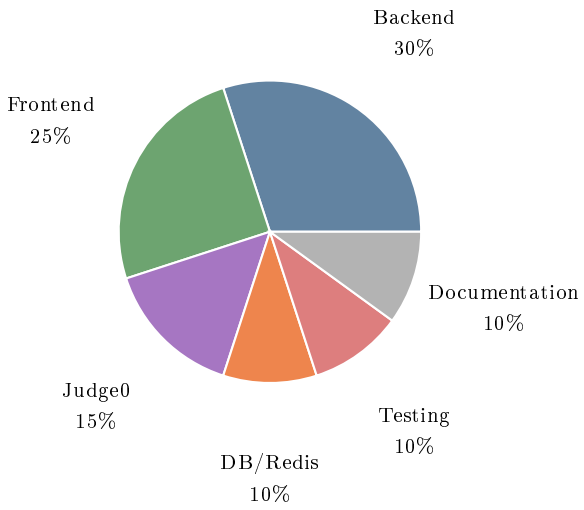


Figure 8.1: Distribution of development effort across project activities

8.2 Language Usage in Battles (Pie Chart)

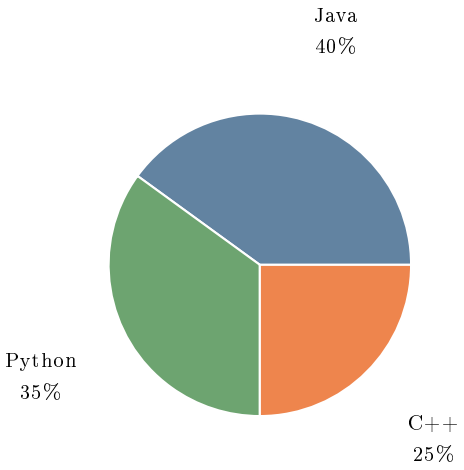


Figure 8.2: Share of programming languages used in test battles

8.3 Judging Time per Language (Bar Chart)

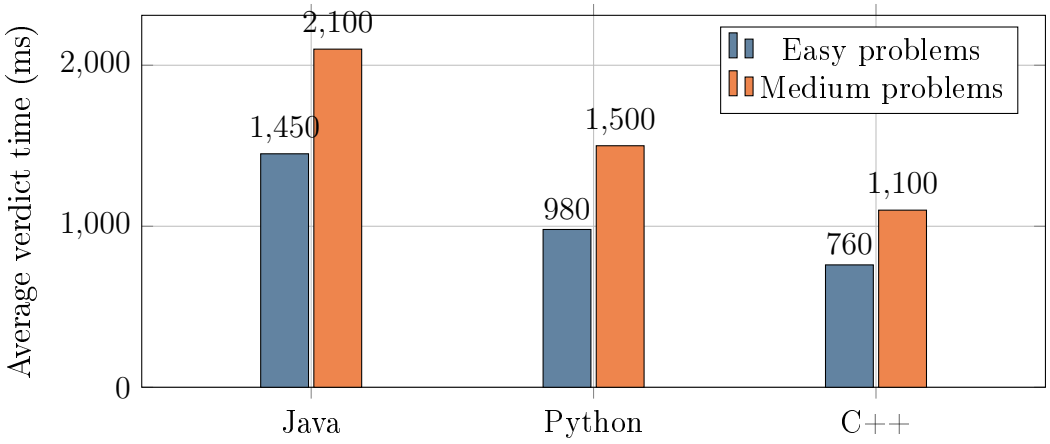


Figure 8.3: Average judging time by language and difficulty

8.4 Matchmaking Wait Time (Line Chart)

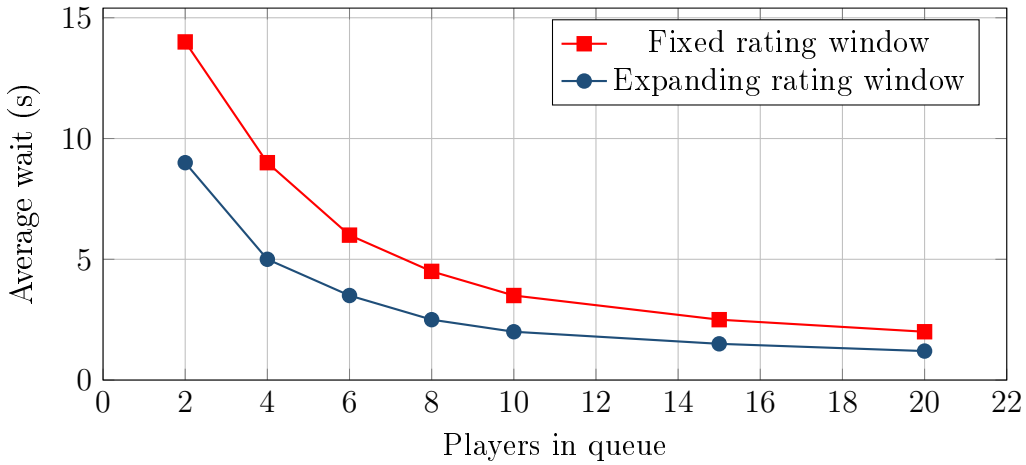


Figure 8.4: Effect of expanding window on matchmaking wait time

8.5 Real-Time Event Latency (Bar Chart)

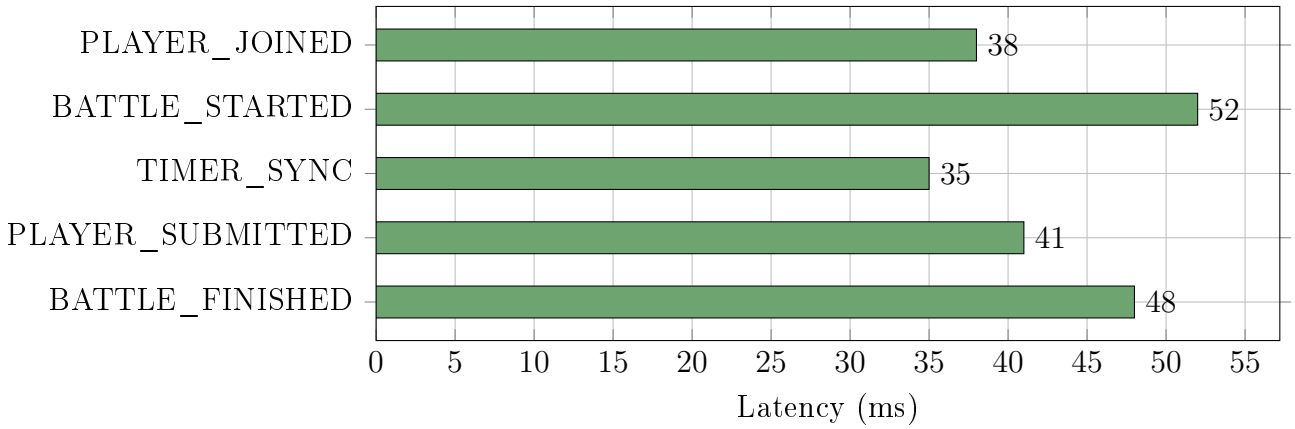


Figure 8.5: Average WebSocket event delivery latency

8.6 Test Results Summary

Module	Tests	Passed	Pass rate
Authentication	12	12	100%
Matchmaking	10	10	100%
Battle / WebSocket	14	13	93%
Judging	16	16	100%
Rating / Winner	10	10	100%
Security	12	12	100%
Total	**74**	**73**	**99%**

Table 8.1: Summary of testing results (sample data)

8.7 Discussion

The results show that server-pushed WebSocket events keep both players synchronised with very low latency, and that the expanding rating window substantially reduces waiting time when the player pool is small without sacrificing match quality for large pools. Running code in Judge0 adds latency, but this is an acceptable trade-off for safe execution.

8.8 Limitations

- Judging throughput is bounded by the number of Judge0 workers.
- The problem set must be curated manually.
- Anti-cheating is basic; external assistance cannot be fully prevented.
- Elo ratings are less reliable for players with few matches.

Chapter 9

Conclusion and Future Scope

9.1 Conclusion

CodeDuel demonstrates that a real-time, competitive programming experience can be built with widely available open-source technologies. The project integrates a reactive front end, a layered Spring Boot back end, real-time WebSocket communication, Redis-backed matchmaking, secure sandboxed execution and a rating system into a cohesive product. The design principle of never trusting the client, together with container isolation, provides a secure basis for running untrusted code. The phased approach shows how a complex distributed application can be delivered incrementally.

9.2 Future Scope

- Tournament and league modes; daily challenges; achievements.
- Spectator mode and battle replay from stored events.
- Advanced anti-cheat: plagiarism similarity detection and tab-switch monitoring.
- Team battles and friend system.
- Horizontal scaling with Redis Pub/Sub and Kubernetes.
- AI-based hint and post-battle feedback system.

Appendix A

Project Schedule (Gantt Chart)

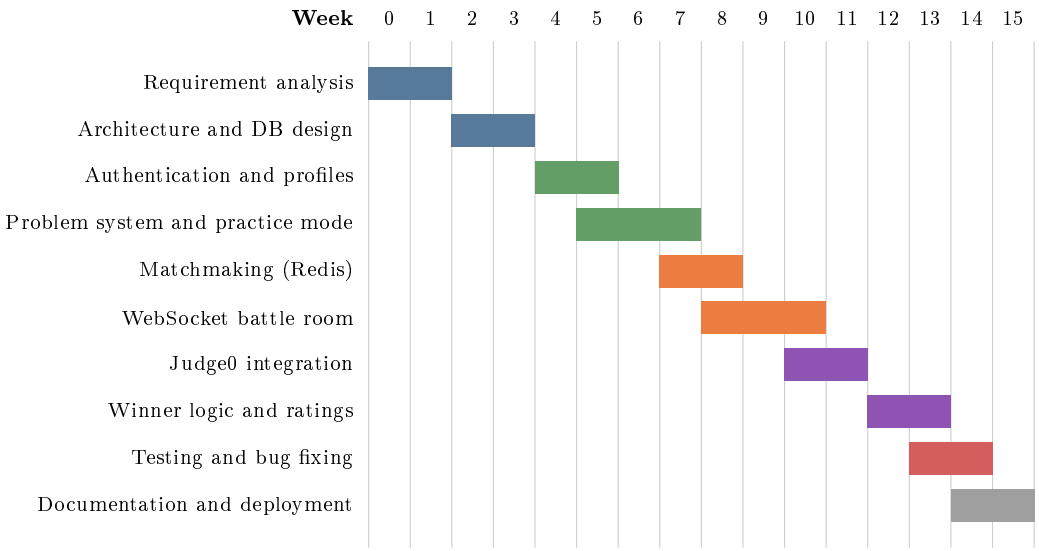


Figure A.1: Project timeline (16 weeks)

Appendix B

Sample Problem Definition

```
title:          Two Sum
difficulty:     Easy
description:    Given an array of integers nums and an integer target,
                return indices of the two numbers that add up to target
                   .
constraints:    2 <= nums.length <= 10^4
visible tests: 3       hidden tests: 10
time limit:     2 s     memory limit: 256 MB
example:        nums = [2,7,11,15], target = 9  ->  [0,1]
```

Listing B.1: Example problem record: Two Sum

Bibliography

[1] Spring Framework Documentation – WebSocket and STOMP Messaging, https://docs.spring.io/spring-framework/reference/web/websocket.html.

[2] Spring Boot Reference Documentation, https://docs.spring.io/spring-boot/.

[3] Spring Security Reference, https://docs.spring.io/spring-security/reference/.

[4] React Documentation, https://react.dev.

[5] Monaco Editor, Microsoft, https://microsoft.github.io/monaco-editor/.

[6] Judge0 – Open Source Online Code Execution System, https://judge0.com.

[7] Redis Documentation, https://redis.io/docs/.

[8] PostgreSQL Documentation, https://www.postgresql.org/docs/.

[9] Docker Documentation, https://docs.docker.com.

[10] A. E. Elo, *The Rating of Chessplayers, Past and Present*, Arco Publishing, 1978.

[11] M. Jones, J. Bradley, N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.

[12] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, 2011.